# Traiter les données en parallèle



# Rappel

01

### Hadoop

Un framework distribué en 4 modules : Common, HDFS, YARN, MapReduce.

02

### HDFS

Découpe les gros fichiers en blocs répartis et répliqués. NameNode = la carte, DataNodes = les données.

03

### Pannes

La tolérance aux pannes est native : une machine tombe, les copies prennent le relais.

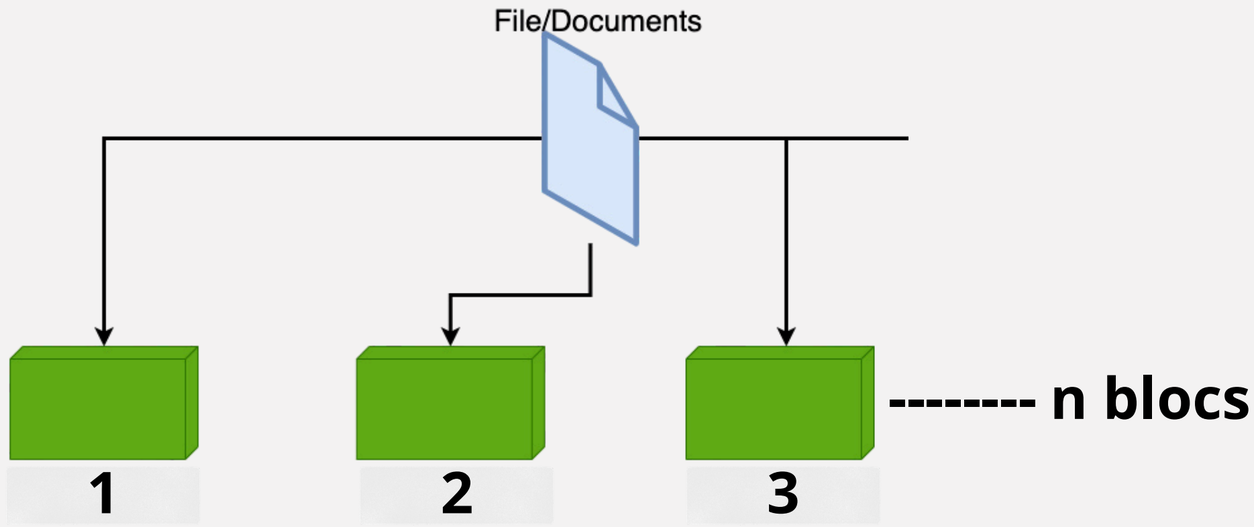
**On sait stocker 1 To sur n machines. Comment le traiter sans le rapatrier sur une seule ?**

# La question d'accroche

1 To de logs, réparti en blocs sur 100 machines. Objectif : compter combien de fois chaque mot apparaît.

Tout rapatrier sur un PC ? Impossible : ça ne tient pas, et déplacer 1 To sur le réseau est très lent.

**Comment faire travailler les “n” machines en même temps, chacune sur les données qu'elle possède déjà ?**

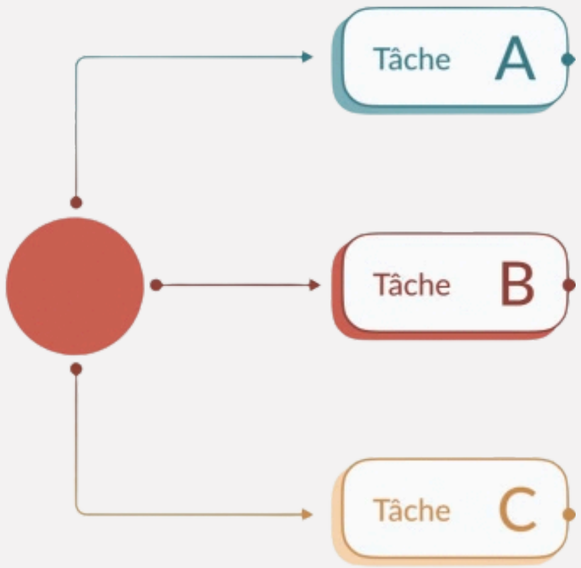


# Objectifs de la séance

| | | |
|---|---|---|
| 01 | **Expliquer** | le principe « diviser pour régner » appliqué au calcul distribué |
| 02 | **Décrire** | les phases Map, Shuffle & Sort, Reduce |
| 03 | **Décomposer** | un problème en couples clé-valeur |
| 04 | **Écrire** | un mapper et un reducer en Python (Hadoop Streaming) |
| 05 | **Tester** | un job en local, le lancer sur le cluster, lire son résultat dans HDFS |
| 06 | **Situer** | MapReduce face à une base relationnelle (batch vs transactionnel) |

PARTIE 1

# Les concepts




# Découper, paralléliser, rassembler

Découper la tâche en petites tâches indépendantes
Les exécuter en parallèle sur plusieurs travailleurs
Rassembler les résultats partiels en un résultat final

**HDFS a déjà découpé le fichier : chaque machine traite le bloc qu'elle possède.**

# L'analogie du dépouillement

10 000 bulletins de vote à compter.

Seul

### Un par un

Vous comptez les 10 000 bulletins vous-même. C'est long.

Map

### Dix paquets

10 personnes comptent chacune leur paquet : « candidat A : 340 ».

Shuffle

### Regroupement

On rassemble tous les partiels d'un même candidat.

Reduce

### Total

Une personne additionne les partiels de chaque candidat.

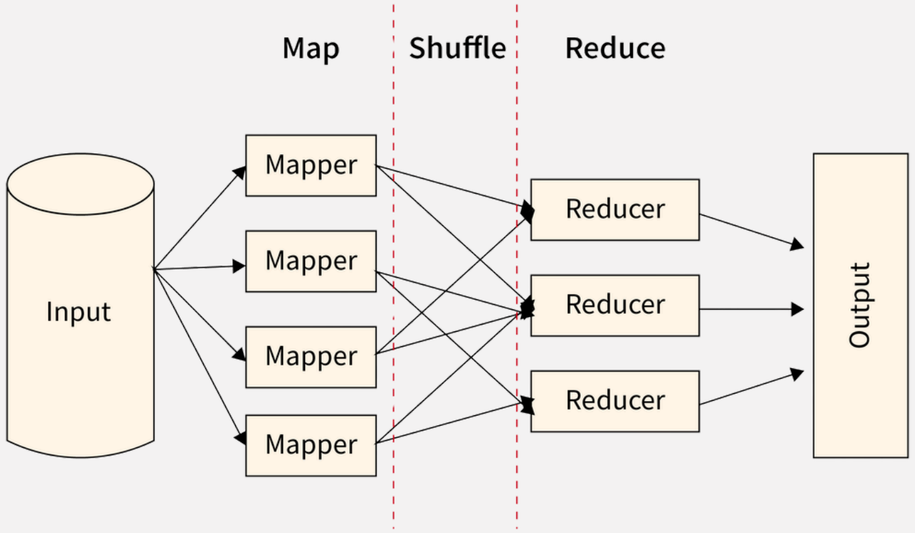
**MapReduce, c'est exactement ça, automatisé sur des machines.**

# Définition

Un modèle de programmation pour traiter le Big Data en parallèle sur plusieurs nœuds.

| | |
|---|---|
| **Vous écrivez** | map et reduce |
| **Hadoop gère** | répartition, pannes, transferts |

**On pense « quoi calculer », pas « comment paralléliser ».**



# Les deux fonctions : Map et Reduce

### La tâche Map

Transforme les données en couples clé-valeur. S'exécute en parallèle sur chaque bloc.

### La tâche Reduce

Prend la sortie de Map, regroupée par clé, et la combine en un ensemble plus petit. Toujours exécutée après les mappers.

**La règle d'or**



# La procédure complète

01

## Input

Lecture et découpage ; chaque enregistrement devient un couple clé-valeur

→

02

## Map

Chaque mapper traite ses enregistrements et émet des couples (clé, valeur)

→

03

## Shuffle & Sort

Regroupe les valeurs par clé et trie. Automatique, coûteux en réseau

→

04

## Reduce

Reçoit une clé et toutes ses valeurs, produit le résultat agrégé

→

05

## Output

Les résultats sont écrits dans HDFS

---

**Option : Combiner** Un mini-reduce local pour réduire le volume transféré.

# Le principe clé-valeur

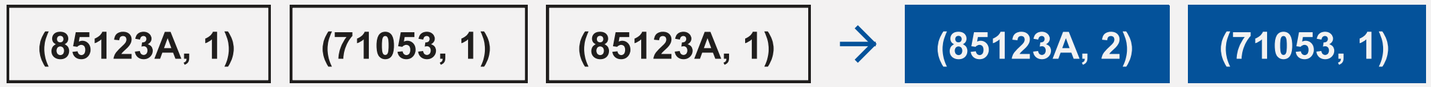
### Tout est couple (clé, valeur)

Concevoir un job, c'est surtout bien choisir la clé et la valeur.

### La clé décide du regroupement

Choisir la clé, c'est choisir sur quoi on agrège avant le Reduce.

**Ventes par produit** clé = StockCode, valeur = 1



# Le comptage de mots (WordCount)

Entrée : « le chat mange le poisson »

Étape 1

## Map

(le, 1)

(chat, 1)

(mange, 1)

(le, 1)

(poisson, 1)

Étape 2

## Shuffle & Sort

(chat, [1])

(le, [1, 1])

(mange, [1])

(poisson, [1])

Étape 3

## Reduce

(chat, 1)

(le, 2)

(mange, 1)

(poisson, 1)

# 1 To

Exactement le même code compte les mots d'une phrase ou d'un fichier de 1 To sur 1 000 machines. C'est ce code que vous écrirez en partie 2.

EXERCICE 1 · 15 MIN · EN BINÔME

# Décomposer en couples clé-valeur

EXERCICE 1

# Trois problèmes sur online_retail.csv

Colonnes : InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country. Montant = Quantity × UnitPrice.

01 Nombre de ventes par produit (StockCode)

02 Somme et nombre des montants par produit (pour un panier moyen)

03 Chiffre d'affaires par pays (Country)

**Pour chacun : que émet Map (clé, valeur) ? Que fait Reduce des valeurs regroupées ?**

# Le Combiner : moins de trafic réseau

### Sans Combiner

(le, 1) (le, 1) (le, 1)

3 couples envoyés sur le réseau

### Avec Combiner

(le, 3)

1 couple, pré-agrégé localement après le Map

**Précaution** Valable pour les opérations associatives et commutatives (somme, max, min). Pas pour la moyenne directement.

# La localité des données

Déplacer 1 To est lent ; déplacer un petit programme est instantané. Chaque mapper tourne sur la machine qui stocke déjà son bloc HDFS.

**« Move computation to data, not data to computation. »**

# MapReduce face au SGBDR

| | BASE RELATIONNELLE (SGBDR) | HADOOP MAPREDUCE |
|---|---|---|
| Taille des données | Gigaoctets | Pétaoctets |
| Accès | Interactif, temps réel | Batch (par lots) |
| Mises à jour | Lecture et écriture fréquentes | Write once, read many |
| Schéma | Statique | Dynamique |
| Montée en charge | Non linéaire | Linéaire (ajouter des machines) |

**Deux outils, deux usages : la transaction rapide d'un côté, l'analyse massive en différé de l'autre.**

# Forces et limites

## Forces

Simple à raisonner

Passe à l'échelle presque linéairement

Tolérant aux pannes

Traite tout type de données

## Limites

Écrit beaucoup sur disque entre les étapes : lent pour l'itératif (machine learning)

Batch seulement

Verbeux à programmer

**Apache Spark** garde les données en mémoire et reprend les idées de MapReduce.

# Synthèse des concepts

Deux fonctions (map, reduce) et une phase automatique de Shuffle & Sort

Tout est couple clé-valeur : choisir la clé, c'est choisir sur quoi on agrège

Le calcul vient à la donnée ; le framework gère parallélisme et pannes

**Vous savez penser en MapReduce. Passons à la pratique.**

PARTIE 2

# En pratique : écrire et lancer un vrai job

# Écrire en Python avec Hadoop Streaming

MapReduce natif s'écrit en Java : lourd pour débuter.

**Hadoop Streaming** est un utilitaire d'Apache Hadoop qui permet de créer et d'exécuter des programmes MapReduce avec n'importe quel langage de programmation (comme Python, Perl ou C++) en utilisant les flux standard (entrées/sorties standard STDIN et STDOUT)




---

**Convention** “clé” et “valeur” , séparées par \t

# Le mapper : mapper.py

```
#!/usr/bin/env python3
import sys

for ligne in sys.stdin:
    for mot in ligne.strip().split():
        print(f"{mot}\t1")  # clé <TAB> valeur
```

Lit chaque ligne et émet un couple (mot, 1) par mot.

**Le mapper ne compte rien : il émet. C'est le reducer qui totalise.**

Chaque mapper tourne en parallèle sur un bloc.

# Le reducer : reducer.py

```
#!/usr/bin/env python3
import sys
mot_courant = None
total = 0
for ligne in sys.stdin:
    mot, valeur = ligne.strip().split("\t")
    valeur = int(valeur)
    if mot == mot_courant:
        total += valeur
    else:
        if mot_courant is not None:
            print(f"{mot_courant}\t{total}")
        mot_courant = mot
        total = valeur
if mot_courant is not None:  # dernier mot
    print(f"{mot_courant}\t{total}")
```

Hadoop trie les couples par clé : les lignes d'un même mot arrivent à la suite.

**On s'appuie sur une entrée déjà triée, grâce à Shuffle & Sort.**

Dès qu'on change de mot, on écrit le total du précédent.

# Tester en local avant le cluster

COMMANDE

```
echo "le chat mange le poisson le chat dort" > texte.txt
cat texte.txt | python3 mapper.py | sort | python3 reducer.py
```

ATTENDU

```
chat    2
dort    1
le      3
mange   1
poisson 1
```

**Le sort local imite Shuffle & Sort.**

# Déposer dans HDFS et lancer le job

```
# 1) Déposer l'entrée dans HDFS
hdfs dfs -mkdir -p /user/etudiant/mr_input
hdfs dfs -put -f texte.txt /user/etudiant/mr_input/
# 2) Lancer le job (Hadoop Streaming)
mapred streaming \
    -input  /user/etudiant/mr_input \
    -output /user/etudiant/mr_output \
    -mapper "python3 mapper.py" \
    -reducer "python3 reducer.py" \
    -file mapper.py -file reducer.py
```

---

**Piège n°1** Le dossier de sortie ne doit pas exister :

hdfs dfs -rm -r /user/etudiant/mr_output

# Lire le résultat

FICHIERS PART-00000, PART-00001…

```
hdfs dfs -ls  /user/etudiant/mr_output
hdfs dfs -cat /user/etudiant/mr_output/part-*
```

---

Attendu : les mêmes comptes qu'au test local.

**Vous venez de lancer un vrai job MapReduce distribué.**

# Le même principe sur online_retail.csv

01

### Changer la clé

Le mapper lit le CSV et émet (Country, 1). Le reducer est le même : il additionne par clé.

02

### Attention CSV

Un split(",") naïf casse si Description contient des virgules. Utiliser le module csv de Python.

03

### Aller au bout

TP Guidé MapReduce, pas à pas. TP à rendre : WordCount sur Description, comptage par produit et pays.

**Mapper CSV**



# Synthèse générale

Comment faire travailler mille machines en même temps sur des données réparties ?

Deux fonctions, map et reduce, écrites ici en Python via Hadoop Streaming

On teste en local (… | sort | …), puis on lance sur le cluster (mapred streaming)

Le framework gère découpage, Shuffle & Sort, parallélisme, pannes, et écrit dans HDFS

Le même code marche sur une phrase et sur des millions de lignes

# À retenir

01

### Le modèle

Map + Reduce + Shuffle & Sort automatique. Tout est clé-valeur.

02

### Streaming

Mapper et reducer en Python, convention clé<TAB>valeur.

03

### Le réflexe

Tester en local avant le cluster. Le dossier de sortie ne doit pas exister.

**Prochaine étape** Comment le cluster orchestre les jobs et partage ses ressources : YARN, puis l'écosystème (Hive, Spark).

EXERCICE 2 · 30 MIN · ÉVALUATION SUR 10

# Vrai ou faux, écrire et lancer

# Vrai ou faux justifié

01 Hadoop Streaming permet d'écrire le mapper et le reducer en Python.

02 En Streaming, la clé et la valeur sont séparées par une virgule.

03 Le dossier de sortie HDFS peut déjà exister quand on lance le job.

04 Tester en local avec … | sort | … imite la phase Shuffle & Sort.

05 Le reducer reçoit les valeurs déjà regroupées par clé.

# Écrire et lancer

ventes_extrait.csv : chaque ligne est pays,montant. Objectif : le chiffre d'affaires par pays.

01 Écrivez le mapper : quelle clé, quelle valeur ?

02 Indiquez ce que fait le reducer.

03 Donnez la commande de test en local.

04 Donnez la commande pour lancer le job sur le cluster.

# Merci

À faire : TP Guidé MapReduce, puis TP à rendre sur online_retail.csv. Prochaine séance : YARN.